

Anteater Poker Game (Version 1.0)

Software Specification

By All In

Members: Sanaa Bebal, Ava Chinn, Alonso De La Cruz, Estrella Distancia Zamora, Lauren Howe, Samuel Kim, and David Lee

Created for the class EECS 22L at University of California, Irvine

Table of Contents

Table of Contents	2
Glossary	3
Game	3
Betting Stages in a Hand (In Chronological Order)	4
Hand Rankings	4
Moves	5
1. Software Architecture Overview	5
1.1 Main Data Types and Structures	5
1.2 Major Software Components	7
1.3 Module Interfaces	7
1.4 Overall Program Control Flow	13
1.5 Automated Client: Poker Bot	13
2. Installation	14
2.1 System requirements and compatibility	14
2.2 Setup and configuration	14
2.3 Building, compilation, and installation	15
3. Documentation of Packages, Modules, and Interfaces	15
3.1 Detailed Description of Data Structures	15
3.2 Detailed Description of Functions and Parameters	16
3.3 Detailed Description of Communication Protocol	23
4. Development Timeline	23
4.1 Partitioning of Tasks	23
4.2 Team Member Responsibilities	24
5. References and External Libraries	24
Contact Information	24
Terms and Conditions	25
Copyright	25
Disclaimer of Warranty	25
Index	25

Glossary

(of terms used in the implementation)

Game

- **Anteater Poker**—a version of poker based on Texas Hold'em, with the addition of the anteater cards and their rules (see “anteater card”)
- **Anteater card**—two exist in a deck; not a part of any standard suit; only a ranked hand when a player holds two anteaters in the hole cards, in which case it ranks between a royal and straight flush and four of a kind
- **Board**—a term used to refer to the visible community cards
- **Blinds**—the bids made by the player one left (the small blind) and two left (the big blind) of the dealer; small blind is 1 point and big blind is 2 points
- **Community cards**—the cards visible to all players that can be used by all players to create a five-card hand
- **Dealer**—the player in charge of giving cards to players; dealer deals clockwise
- **Five-card hand**—a group of five cards comprised of some combination of the two hole cards and the five community cards
- **Hand**—refers to the complete sequence of moves made within the repeatable segment of the game, starting with players receiving their cards and ending with someone winning the hand
- **Hole cards**—the two cards held by the individual player; only the player knows what their cards are and only the player can use them to create a five-card hand
- **Kicker**—refers to the cards that are not part of the ranked part of a hand that are used to break ties; used for ties involving one pair, two pair, three of a kind, and four of a kind
- **Pot**—holds the main points bet in a hand
- **Round**—consists of dealing cards and multiple betting intervals where the players match the highest bet, raise, or fold. Rounds include pre-flop, flop, turn, and river betting.
- **Side pot**—not the main pot, but a pot used when at least one player goes all in and the other players continue betting; cannot be won by the player(s) who went all in
- **Suit**—a group of thirteen cards with values ranging from two to ace; the four types ranked lowest to highest are clubs, diamonds, hearts, and spades
- **Value**—refers to the second attribute in addition to suit that a card has; the fourteen values are 2-10, Jack, Queen, King, Ace, and Anteater
- **Game end**—game automatically ends when one player wins all the points or when all players agree to end the game via vote (computer players get no vote)

Betting Stages in a Hand (In Chronological Order)

1. **Preflop**—no cards on the board; betting starts with the small blind and the big blind, then proceeds to the player left of the big blind; each player can call, raise, or fold until all players have either met the highest bet made or folded
 - a. Note: During this stage, the only player who can check is the player who made the big blind, and this only occurs when everyone else has called or folded (i.e. not raised); the big blind player can then choose to either check (ending the preflop round) or raise (continuing the round, as all players then have to call, raise, or fold)
2. **Flop**—three cards on the board; each player can check (if no one else has bet), bet, call, raise, or fold until all players have either met the highest bet made or folded
3. **Turn** - After the flop, the dealer reveals the fourth community card; often where draws are complete and the pot size increases dramatically to force decisions
4. **River** - The last community cards is dealt in a game and where the final betting round begins
5. **Showdown** - the final stage of a hand in which all betting has concluded and only occurs when there are at least two players remaining after the final betting round or checking on the river. Typically, the last player to make a bet or raise on the final round is the first to reveal their hand, but if someone checks, the player to the left of the dealer reveals their hand first.

Note: If at any point all but one player has folded in the rounds leading up to the showdown, the one player left wins the pot.

Hand Rankings

1. **Royal flush**—the player has the 10, Jack, Queen, King, and Ace of all one suit
2. **Straight flush**—the player has five cards of consecutive value all of the same suit (i.e. a straight all in the same suit)
3. **Anteater Pair**—the player holds two anteaters in the hole cards
4. **Four of a kind**—the player has four cards of the same rank and one of another rank
5. **Full house** - the hand has three cards of one rank and two cards of another rank
6. **Flush** - a hand contains five cards of sequential order, not necessarily in consecutive order
7. **Straight** - a five-card hand containing sequential ranks, not considering the suits
8. **Three of a kind** - consisting of three cards of the same rank with two unmatched cards
9. **Two pair** - contains two cards of one rank, two cards another rank, and one card of a third rank
10. **Pair** - consists of two cards of the rank and three unrelated or unmatched cards
11. **High card** - contains none of the hands above, the hand is determined by the highest ranking card

The royal flush is the highest-ranking hand possible in the game

Moves

- **All-in**– Wager all of a player's remaining points on a single hand. The player can't take any more betting actions unless they win additional points in any side pots.
- **Bet**– The first voluntary wage made in a betting round.
- **Call**– Match the current highest bet made by the previous player. Only available when a preceding player has made a bet or raise during a betting round.
- **Check**– Pass the action to the next player without wagering. Only available if no player has bet before you in that round.
- **Fold**– Discarding your current hand and forfeit any points contributed in the pot
- **Raise**– Increase the current highest bet, requiring other players to match the new amount to keep their hand. Only available after a bet has been made.

1. Software Architecture Overview

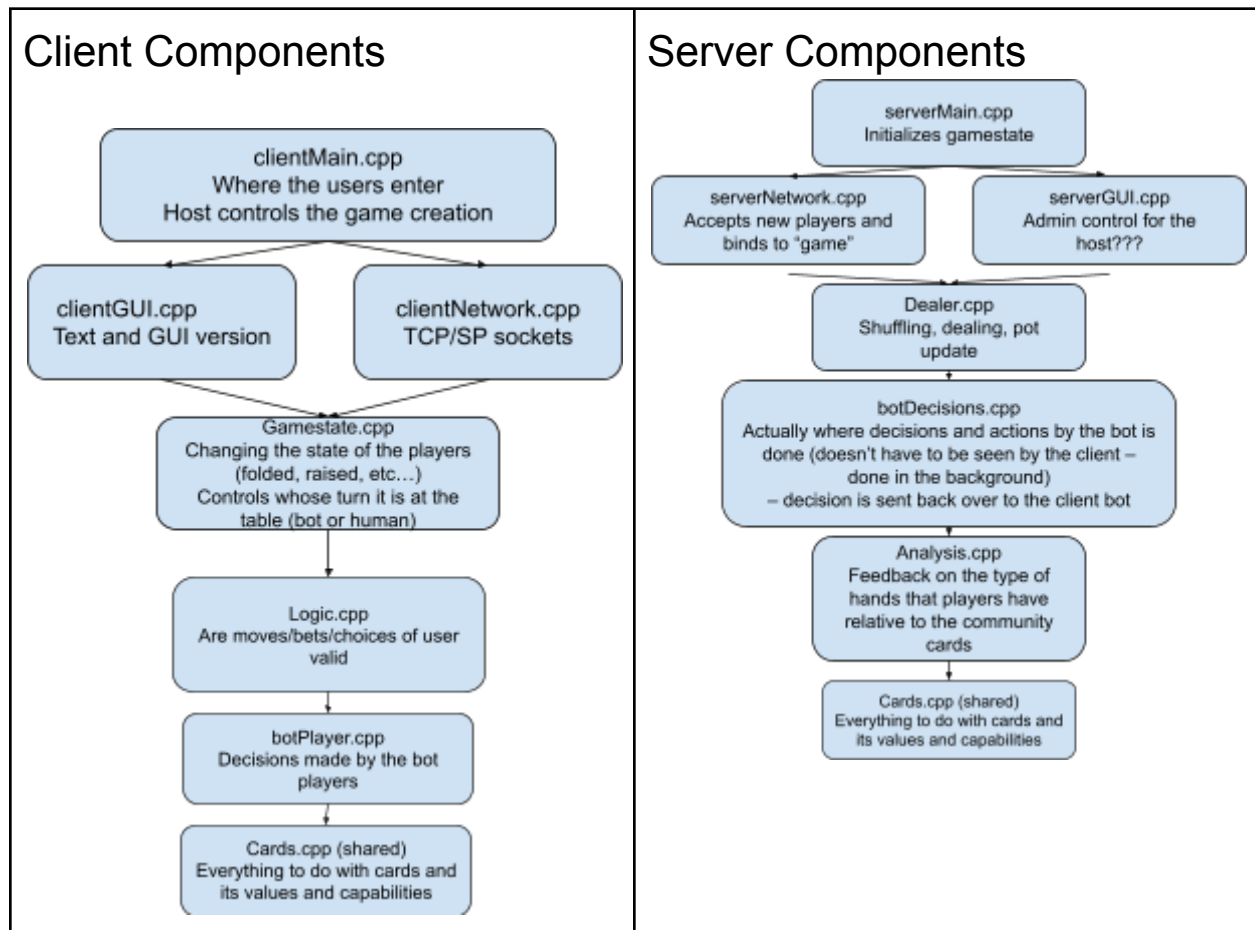
1.1 Main Data Types and Structures

- Enumerators:
 - enum suit–represents Hearts, Clubs, Diamonds, Spades, and Anteaters
 - enum val–represents Two-Ten, Jack, Queen, King, Ace, and Anteater
 - enum Round–represents Preflop, Flop, Turn, River, and Showdown
 - enum PlayerType–represent Human (1) and Computer (2) (players)
 - enum SpecialBets–assigning Check to 0 and Fold to -1
 - enum PackageType–delineates between the types of data sent between the client and server (login and game)
 - **Note:** Although not an enum, player numbering starts at 0
- Structures:
 - struct gameState
 - int packageType = game // used for sending data via the buffer between the client and the server (see PackageType enum)
 - int numPlayers // holds the number of players in the game
 - int round // holds the round number (Preflop, Flop, Turn, River, or Showdown)
 - int dealerPlayer // holds which player is the “dealer” (note that dealing is done by the server, not the player)
 - PILES allCards // holds all the player hands, the community cards, and the leftover cards (in that order)
 - int pot // represents the amount of money in the pot

- int callAmount // the amount of money needed to call in the round
 - PLAYERS players // holds player information
 - int playerTurn // holds which player's turn it is
 - struct Player
 - int playerNum // holds player number
 - char *name // holds the player's name
 - int playerType // holds if player is Human (1) or Computer (2) (see PlayerType enum)
 - int score // holds a player's current score (does not include points that have been bet during the hand)
 - int bet // holds a player's bet for the current round of betting
 - int isInHand // still playing in the hand or not (i.e. 1 if they haven't folded, 0 if they have)
 - int isEliminated // should be removed from the list if this is true
 - int playerSocket // this struct can be removed if struct Player is only used by the server & int playerSocket can be moved back into the struct
 - struct LoginInfo
 - int packageType = login // used for sending data via the buffer between the client and the server (see PackageType enum)
 - int isHost // represents if the player is the "host" player (the host determines the number of players to enter the game, the number of bots, etc.)
 - char *playerName // holds the player's name
 - int playerNum // represents the player number at the table (numbering starts at zero)
 - char *password // set by the player if the host (from client to server), or entered by a joining player if not the host (from server to client)
 - int playerType (Human or Computer (see PlayerType enum))
 - int numPlayers // number of players in the room
 - PLAYERSFOUND playersFound // 0 if player is not found; 1 or 2 if player is Human / Computer (see PlayerType enum)
 - typedef struct LoginInfo LOGININFO;
 - typedef struct playerState GAMESTATE;
 - typedef struct Player PLAYER;
- Lists (vectors):
 - typedef std::vector<PLAYER> PLAYERS; // used to hold info about the players
 - typedef std::vector<Card> PILE; // used to hold one of these items: deck, player hands, community cards, leftover cards, etc.
 - typedef std::vector<PILE> PILES; // used to hold player hands, community cards, and leftover cards (in that order)
 - typedef std::vector<unsigned char> BUF; // used to represent the buffer for TCP/IP communication
 - typedef std::vector<int> PLAYERSFOUND; // used in struct LoginInfo (see above)
- Classes:
 - Card class

- Attributes: val, suit
- Methods:
 - Card() and Card(int valVar, int suitVar) [constructors]
 - getValue() and getSuit() [getters]

1.2 Major Software Components



1.3 Module Interfaces

Major functions in each module

clientMain.cpp

Local files required: GUI.hpp, Client.hpp, GameState.hpp, Logic.hpp, botPlayer.hpp, Analysis.hpp, Cards.hpp

Libraries required: <stdio.h>, <stdlib.h>, <string.h>

Internal Functions:

- int main(void)
 - Calls: parsingArguments(), textMenu() or guiMenu(), gameLoop(), closeGame()
- void gameLoop(GAMESTATE gameState)
 - Calls: displayCards(), displayPot(), displayScores(), displayAnalysis(), askAction(), acceptBotDecision()
- void closeGame(GAMESTATE gameState)
 - Calls: error(), displayEndGame()
- GAMESTATE parsingGameArguments(BUF buf)
 - Calls: error()
- LOGININFO parsingLoginArguments(BUF buf)
 - Calls: error()
- BUF createBuffer(GAMESTATE gameState)
 - Calls: error()
- BUF createBuffer(LOGININFO loginInfo)
 - Calls: error()

clientGUI.cpp

Local files required: Gamestate.hpp, Analysis.hpp

Libraries required: <gtk/gtk.h> <string.h> <stdio.h> <vector>

Internal Functions:

- LOGININFO textMenu(void)
 - Calls: N/A
- LOGININFO guiMenu(void)
 - Calls: N/A
- void displayCards(PILES piles)
 - Calls: printCard()
- void displayPot(int pot)
 - Calls: N/A
- void displayScores(PLAYERS players)
 - Calls: N/A
- int askAction(GAMESTATE gameState, int playerNum)
 - Calls: N/A
- int displayAnalysis(PILES piles, int playerNum)
 - Calls: handHierarchy()

clientNetwork.cpp

Local files required: Cards.hpp

Libraries required: <stdio.h>, <stdlib.h>, <unistd.h>, <string.h>, <sys/types.h>, <sys/socket.h>, <netinet/in.h>, <netdb.h>

Internal Functions:

- void error(const char *msg)

- Calls: none
- Int connectClient(int argc, char *argv[])
 - Calls: socket(), connect(), write(), read(), close()

Gamestate.cpp

Local files required: Logic.hpp, Gamestate.hpp, Cards.hpp, Analysis.hpp

Libraries required: <stdio.h>, <stdlib.h>, <vector>

Internal Functions:

- int nextPlayer(GAMESTATE *state)
 - Calls: none
- int nextRound(GAMESTATE *state)
 - Calls: none
- int updatePot(GAMESTATE *state, int amount)
 - Calls: none
- int resetHand(GAMESTATE *state)
 - Calls: shuffleDeck(), dealCards()

Logic.cpp (I/O error handling, reactionary functions to player input)

Local files required: Logic.hpp, Gamestate.hpp

Libraries required: <stdio.h>, <stdlib.h>

Internal Functions:

- int validBet(GAMESTATE state, int playerNum, int amount)
 - Calls: none
- int validCall(GAMESTATE state, int playerNum)
 - Calls: none
- int validCheck(GAMESTATE state, int playerNum)
 - Calls: none
- int validRaise(GAMESTATE state, int playerNum, int amount)
 - Calls: validBet()
- int validFold(GAMESTATE state, int playerNum)
 - Calls: none
- int betting(GAMESTATE *state)
 - Calls: validBet(), validCall(), validCheck(), validRaise(), validFold()

botPlayer.cpp (Decisions made by the bot player)

Local files required: Gamestate.hpp, Logic.hpp

Libraries required: <stdio.h>, <stdlib.h>

Internal functions:

- int acceptBotDecision(GAMESTATE *gameState, int playerNum, int action, int amount)
 - Calls: validBet(), validCall(), validCheck(), validRaise(), validFold()
 - All dependent on what is received from the bot action decision making for the server side, checking whether the decision is allowed before proceeding

serverMain.cpp (initializes gamestate)

Local files required: serverNetwork.hpp, serverGUI.hpp, Dealer.hpp, botDecisions.hpp, Analysis.hpp, Cards.hpp

Libraries required: <stdio.h>, <stdlib.h>, <unistd.h>, <string.h>, <sys/types.h>, <sys/socket.h>, <netinet/in.h>, <netdb.h>

Internal Functions:

- int main(void)
 - Calls: parsingArguments(), createBuffer(), getPlayerMove()
- GAMESTATE parsingArguments(BUF buf, int whichStructure)
 - Calls: error()
- LOGININFO parsingArguments(BUF buf, int whichStructure)
 - Calls: error()
- BUF createBuffer(GAMESTATE gameState)
 - Calls: error()
- BUF createBuffer(LOGININFO loginInfo)
 - Calls: error()
- int getPlayerMove(GAMESTATE gameState, int playerNum)
 - Calls: createBuffer(GAMESTATE), parsingArguments() [returns GAMESTATE]

serverNetwork (allows players to connect to server)

Local files required: none

Libraries required: <stdio.h>, <stdlib.h>, <unistd.h>, <string.h>, <sys/types.h>, <sys/socket.h>, <netinet/in.h>, <netdb.h>

Internal functions:

- void error(const char *msg)
 - Calls: N/A
- void startServer(int argc, char *argv[])
 - Calls: socket(), bzero(), htons(), bind(), listen(), accept(), read(), write(), close()

Cards.cpp (Reshuffles, reinitializes cards in deck; shared between client and server)

Local files required: Cards.hpp, Constants.hpp

Libraries required: <vector>, <algorithm>, <cstdlib>, <stdio.h>, <stdlib.h>

Internal Functions:

- PILE initDeck(void)
 - Calls: N/A
- void printCard(Card card)
 - Calls: N/A
- int printDeck(PILE deck)
 - Calls: N/A
- int printPiles(PILES piles)
 - Calls: printDeck()

serverGUI.cpp (displaying basic game/player info on the server end)

Local files required: Cards.hpp, GameState.hpp

Libraries required: <gtk/gtk.h> <string.h> <stdio.h> <vector>

Internal functions:

- void displayServerGUI(GAMESTATE gameState)
 - Calls: N/A

Dealer.cpp (shuffling, dealing, pot update)

Local files required: Cards.hpp, serverMain.hpp

Libraries required: <stdio.h>, <stdlib.h>, <vector>

Internal functions:

- PILE shuffling(PILE deck)
 - Calls: none
 - Note: srand(time(NULL)) must be used in the main function
- PILES dealing(PILE *deck, int numPlayers, int playerNum)
 - Calls: none
- int updatePot(GAMESTATE *gameState)
 - Calls: none
- int updateRound(GAMESTATE *gameState)
 - Calls: none
- GAMESTATE round(GAMESTATE gameState)
 - Calls: shuffling(), dealing(), getPlayerMove()

botDecisions.cpp (where decisions and actions by the bot is done)

Local files required: Cards.hpp

Libraries required: <stdio.h>, <stdlib.h>

Internal functions:

- int decideAction(GAMESTATE gameState, int playerNUM)
 - Calls: evaluateHand(), calculateOdds(), getValidMoves(), validBet(), validCall(), validCheck(), validRaise(), validFold(), sendToClient()
- int evaluateHand(PILES piles, int playerNum)
 - Calls: handHierarchy()
- int calculatedOdds(int handScore)
 - Calls: none
- void sendToClient(int socket, int playerMove)
 - Calls: network related send/write

Analysis.cpp (Analyzes each active player's hands)

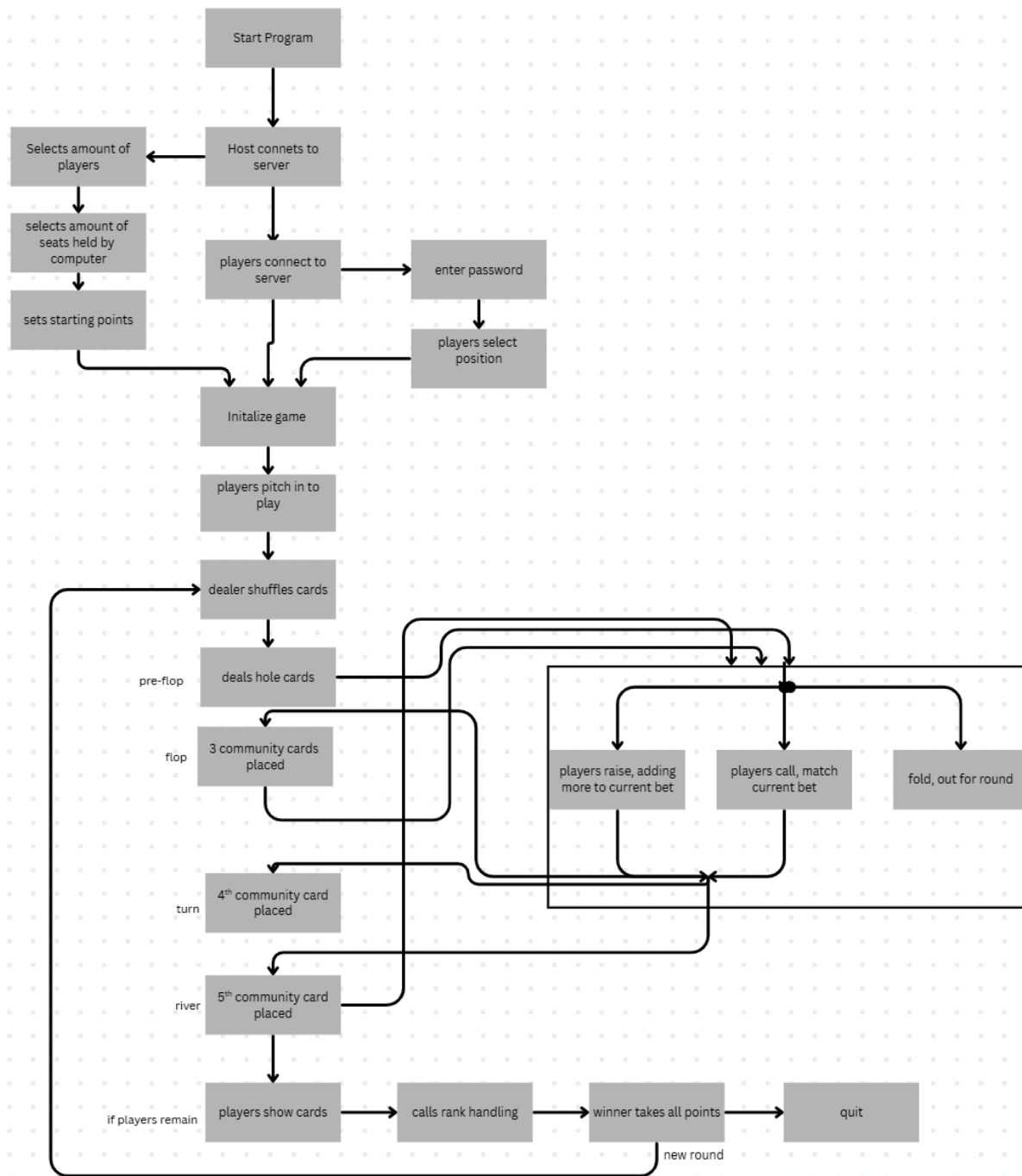
Local files required: Cards.hpp

Libraries required: <stdio.h>, <stdlib.h>

Internal Functions:

- `int handHierarchy(PILES piles, int playerNum)`
 - Functionality: returns the highest hand based on the following functions
 - Calls: All score calculating functions below
- `int royalflush(PILES piles, int playerNum)`
 - Functionality: returns 11000 if possible and 0 if not
 - Calls: N/A
- `int straightflush(PILES piles, int playerNum)`
 - Functionality: returns 10,000 base score + high card tiebreaker bonus if possible and 0 if not
 - Calls: N/A
- `int antpair(PILES piles, int playerNum)`
 - Functionality: return 9,000 if possible and 0 if not
 - Calls: N/A
- `int ofakind(PILES piles, int playerNum)`
 - Functionality: Returns an int in (1000, 2000] for a pair, in (3000,4000] for 3-of-a-kind, and in (7000,8000] for 4-of-a-kind; score is base score + tiebreaker bonus
 - Calls: `kicker()`
- `int fullhouse(PILES piles, int playerNum)`
 - Functionality: returns an int in (6000,7000] if there is a full house and 0 if not (score is base score + bonus determined by values within the ranked hand for tiebreaking purposes)
 - Calls: N/A
- `int flush(PILES piles, int playerNum)`
 - Functionality: returns an int in (5000,6000] if there is a flush and 0 if not (score is base score + tiebreaker bonus)
 - Calls: N/A
- `int straight(PILES piles, int playerNum)`
 - Functionality: returns an int in (4000,5000] if there is a straight and 0 if not (score is base score + tiebreaker bonus)
 - Calls: N/A
- `int twopair(PILES piles, int playerNum)`
 - Functionality: returns an int in (2000,3000] if there is a two pair and 0 if not (score is base score + tiebreaker bonus)
 - Calls: `kicker()`
- `int highcard(PILES piles, int playerNum)`
 - Functionality: returns an int within (0,1000] if there is a high card and 0 if not (score is base score + tiebreaker bonus)
 - Calls: N/A
- `int kicker(PILE unusedHand, int whichKicker)`
 - Functionality: returns the enum value of a card to indicate a kicker value (highest unused card for first kicker, second highest for second, etc.)

1.4 Overall Program Control Flow



1.5 Automated Client: Poker Bot

For our automated client, the bot players and all decision-making will occur on the server side, in the `botDecisions.cpp`, while the client side, in the `botDecisions.cpp` receives the bot's actions and then applies them to the client display or application. The bot itself remains as a separate client process that connects to the game server, similarly to a human-controlled client.

In general, the automated player is just another participant in the poker game, replacing a human user. Whenever there are extra seats to fill, or a host versus computer, depending on the host user's specifications for the current game, the bot players are easily identifiable through the `PlayerType` enum, which distinguishes between human and computer players within the shared game state.

The server-side controls all bot decision-making: based on the current game state, the bot evaluates it, and a legal action is then decided for the selected bot player. Relevant functions like `decideAction()`, `evaluateHand()`, and `calculateOdds()` all support the bot's actions, such as check, call, raise, or fold. From there, the server sends the selected action to the bot's client application, which then processes it like another human client would.

Since each bot operates as its own client, communication with the server must be maintained to update information and respond only when it is the given bot player's turn. The design is intended to keep the bot separate and allow for both the human and computer to coexist in the game environment without excessive special-case handling.

2. Installation

2.1 System requirements and compatibility

The user must satisfy the following conditions to correctly install the software:

- **Operating System:** RHEL8 64-bit Linux
- **Compiler:** GNU Compiler Collection (gcc)
- **Build System:** GNU *make*
- **Libraries:** Standard C libraries; GTK 3.0
- **Utilities:** tar, gzip, evince, and shell environment
- **Hardware Requirements:** Minimal CPU and memory resources sufficient for compiling and executing.
- **Server Access:** Must be able to access the UCI EECS Servers

2.2 Setup and configuration

Using the command line, go into the installed folder and run the following code:

```
A. Extract the Source Package
gtar xvzf Poker_V1.0_src.tar.gz
cd poker
```

B. Build the Software

make (from within the poker directory)

C. Run the Executable

bin/poker (from within the poker directory)

D. Remove compiled object files

make clean (from within the poker directory)

2.3 Building, compilation, and installation

A. Create the Binary Distribution

```
tar -zcvf Poker_V1.0.tar.gz \  
    ./doc/README \  
    ./doc/COPYRIGHT \  
    ./doc/Poker_UserManual.pdf \  
    ./doc/INSTALL \  
    ./bin/poker \  
    ./bin/*.bmp \  
    ./Makefile
```

B. Create the Source Distribution

```
tar -zcvf Poker_V1.0_src.tar.gz \  
    ./doc \  
    ./bin \  
    ./src \  
    ./Makefile
```

C. Purpose of Each Package

Poker_V1.0.tar.gz - Contains the executable and documentation for end users

Poker_V1.0_src.tar.gz - Contains full source code for developers and maintainers

3. Documentation of Packages, Modules, and Interfaces

3.1 Detailed Description of Data Structures

Detailed descriptions of enumerators, typedefs, and structures can be found in Section 1.1.

Their actual code will be similar to their setup within this section.

The Card class, however, is included below because its details and setup go beyond what was explained in Section 1.1.

Card class (cards.hpp):

```
class Card{
public:
    int val;
    int suit;

    Card(void) { // overloading
        val = -1;
        suit = -1;
    }
    Card(int valVar, int suitVar){
        val = valVar;
        suit = suitVar;
    }

    int getValue(void) const { // const means funct can't modify
anything--good for all getters
        return val;
    }
    int getSuit(void) const{
        return suit;
    }
};
```

3.2 Detailed Description of Functions and Parameters

Note: There will also be unit and system testing files in the final version. For brevity purposes, they are not included in these descriptions.

clientMain.cpp -

Provides: where main game flow is run and where all functions are called upon

Requires:

int main(void)

- Arguments: none
- Result: runs the client interface that includes the client GUI, prompts players when it is their turn, and sends a valid move to the server

void gameLoop(GAMESTATE gameState)

- Arguments: GAMESTATE structure containing the initial state of the game (determined by the login information of all players)

- Result: runs the client aspects of the game, looping through and prompting the user when necessary

void closeGame(GAMESTATE gameState)

- Arguments: GAMESTATE structure specifying the final state of the game
- Result: shows the final state of the game and allows players to exit the game

GAMESTATE parsingGameArguments(BUF buf)

- Arguments: BUF vector containing the buffer sent by the server
- Result: returns a GAMESTATE structure based on the buffer information sent by the server

LOGININFO parsingLoginArguments(BUF buf)

- Arguments: BUF vector containing the buffer sent by the server
- Result: returns a LOGIN structure based on the buffer information sent by the server

BUF createBuffer(GAMESTATE gameState)

- Arguments: GAMESTATE structure
- Result: turns a GAMESTATE structure into a BUF vector that acts as a buffer used to send information to the server

BUF createBuffer(LOGININFO loginInfo)

- Arguments: LOGININFO structure
- Result: turns a LOGININFO structure into a BUF vector that acts as a buffer used to send information to the server

clientGUI.cpp -

Provides: text and GUI application

Requires:

LOGININFO textMenu(void)

- Arguments: none
- Result: returns a structure with login info

LOGININFO guiMenu(void)

- Arguments: none
- Result: returns a structure with login info

void displayCards(PILES piles)

- Arguments: vector containing hand/board/extra card vectors
- Result: shows to users the cards they have

void displayPot(int pot)

- Arguments: the current pot of the game
- Result: displays the pot to the user

void displayScores(PLAYERS players)

- Arguments: vector containing hand/board/extra card vectors
- Result: shows the user the players' scores

int askAction(GAMESTATE gameState, int playerNum)

- Arguments: vector containing hand/board/extra card vectors and the general state of the game
- Result: displays the prompting to the user to ask for their next move

int displayAnalysis(PILES piles, int playerNum)

- Arguments: vector containing hand/board/extra card vectors
- Result: displays the likelihood of a person winning the game

void displayEndGame(PILES piles, int playerNum)

- Arguments: vector containing hand/board/extra card vectors
- Result: displays the players' end scores

clientNetwork.cpp - TCP/SP Sockets

Provides: connects client to server

Requires:

void error(void)

- Arguments: called when system call fails
- Result: returns error statement

int connectClient(int argc, char *argv[])

- Arguments: none
- Result: connects client to server

gameState.cpp -

Provides: controls the next actions of the game

Requires:

int nextPlayer(GAMESTATE *state)

- Arguments: passes turn to next player still in the game
- Result: returns the updated number for the currentPlayer

int nextRound(GAMESTATE *state)

- Arguments: passes turn to next player still in the game
- Result: returns the updated number of the round of the game

int updatePot(GAMESTATE *state, int amount)

- Arguments: passes turn to next player still in the game
- Result: returns the updated number of the pot amount

Logic.cpp -

Provides: checks whether moves/bets/choices of users are valid

Requires:

int validBet(GAMESTATE gameState, int playerNum, int amount)

- Arguments: checks to see if player entered a valid bet
- Result: returns int 1 if a valid bet (i.e. no one else has made a bet so far in the round) and 0 if not

int validCheck(GAMESTATE gameState, int playerNum)

- Arguments: determines if check is available to player
- Result: returns 1 if check is valid (i.e. no one else has made a bet so far in the round) and 0 if not

int validAllIn(GAMESTATE gameState, int playerNum, int amount)

- Arguments: determines if all in is available to player
- Result: returns 1 if all in is valid (i.e. player has enough points and it is all of their remaining points) and 0 if not

int validFold(GAMESTATE gameState, int playerNum, int amount)

- Arguments: determines if fold is available to player
- Result: returns 1 if fold is valid (primarily checking if round isn't showdown) and 0 if not

int validCall(GAMESTATE gameState, int playerNum, int amount)

- Arguments: determines if call is available to player
- Result: returns 1 if call is valid (i.e. player has enough points) and 0 if not

int validRaise(GAMESTATE gameState, int playerNum, int amount)

- Arguments: determines if raise is available to player and player has enough points
- Result: returns 1 if raise is valid (i.e. player has enough points) and 0 if not

int getValidMoves(GAMESTATE gameState, int playerNum, int amount)

- Arguments: calls functions above to determine what moves are valid
- Result: returns an array with each entry representing a different option

botPlayer.cpp - Decisions made by the bot player

Provides: receives information (decisions of the current bot player) from the server and applies to client's screen and application

Requires:

int acceptBotDecision(GAMESTATE *gameState, int playerNum, int action, int amount)

- Arguments: receives information from serverBot
- Result: updates the client's game state and display to reflect back the bot's decided move; returns 1 for success and 0 for failure

serverMain.cpp- initializes gamestate

Provides: initializes the state of the game based upon the specifications of the host user (will provide info such as how many decks are use, how many player, etc.)

Requires:

int main(void)

- Arguments: N/A
- Result: main function on the server that runs the game

GAMESTATE initGameState(LOGININFO loginInfo)

- Arguments:
- Result: initiates the GAMESTATE structure based on the login information

int runGame(GAMESTATE startGameState)

- Arguments:
- Result:

int handlingVotes()

- Arguments: prompted if a player would like to quit then prompts all other players if they would like to end the game early
- Result: returns 1 if players all players are in agreeance of quitting and 0 otherwise

Cards.cpp - initialization of cards in the deck and shared between client and server

Provides: all the values for each of the specific cards in the deck

Requires:

PILE initDeck(void)

- Arguments: None
- Result: returns an initialized deck

void printCard(Card card)

- Arguments: None
- Result: Prints a card based on its value and suit

int printDeck(PILE deck)

- Arguments: PILE vector containing a list of cards
- Result: can be used to print any PILE, including player hands and the board

int printPiles(PILES piles)

- Arguments: PILE vector containing all of the cards
- Result: returns back the location of cards whether in the players hand, community deck, and in the cards to be drawn

serverNetwork.cpp - allows players to connect to server

Provides: allows clients to connect to server

Requires:

void error(const char *msg)

- Arguments: called when system calls fails
- Result: displays error message

void startServer(int argc, char *argv[])

- Arguments: command-line arguments
- Result: opens up connection to the client

serverGUI.cpp - displaying basic game/player info on the server end

Provides: a display to the host user for relevant game information

Requires:

void displayServerGUI(GAMESTATE gameState)

- Arguments: current gameState of the game
- Result: returns back relevant info to the host user

Dealer.cpp - shuffling, dealing, pot update

Provides: the necessary information and moves for the dealer of the game

Requires:

PILE shuffling(PILE deck)

- Arguments: vector containing the 54 cards initially
- Result: returns back a mixed up 54 card deck

PILES dealing(PILE *deck, int numPlayers, int playerNum)

- Arguments: vector of the current deck, number of player, and player number
- Result: returns back the updated pile of each player

int updatePot(GAMESTATE *gameState)

- Arguments: current gameState of the game
- Result: returns back the updated number of the pot

int updateRound(GAMESTATE *gameState)

- Arguments: current gameState of the game
- Result: returns back the updated number of the updateRound

GAMESTATE round(GAMESTATE gameState)

- Arguments: current gameState of the game
- Result: returns back the updated Gamestate of the round after each move was made

botDecisions.cpp - where decisions and actions by the bot is done

Provides: all logic needed for the bot players to make decisions (all settled actions will be sent to the client to be completed)

Requires:

int decideAction(GAMESTATE gameState, int playerNum)

- Arguments: makes final decision of what move to make
- Result: returns a number that reflects the player's action (bet, call, fold, etc.)

int evaluateHand(PILES piles, int playerNum)

- Arguments: vector containing the piles of cards (community deck, player hands), current player
- Result: returns back scored based upon the community deck

int calculateOdds(int handScore)

- Arguments: determines likelihood of winning based on current hand and flop
- Result: returns back percentage of likelihood of winning

sendToClient(int socket, int playerMove)

- Arguments: what;s the decided player move based upon evaluation
- Result: sends settled actions to client

analysis.cpp - Analyzes each active player's hands

Provides: Scoring for hands (based on showdown round—i.e. all cards visible)

Requires:

int handHierarchy(PILES piles, int playerNum)

- Arguments:
- Result: returns the highest ranked score of a hand

int royalflush(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns 11000 if possible and 0 if not

int straightflush(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns 10,000 base score + high card tiebreaker bonus if possible and 0 if not

int antPair(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: return 9,000 if possible and 0 if not

int ofakind(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: Returns an int in (1000, 2000] for a pair, in (3000,4000] for 3-of-a-kind, and in (7000,8000] for 4-of-a-kind; score is base score + tiebreaker bonus

int fullhouse(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns an int in (6000,7000] if there is a full house and 0 if not (score is base score + bonus determined by values within the ranked hand for tiebreaking purposes)

int flush(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns an int in (5000,6000] if there is a flush and 0 if not (score is base score + tiebreaker bonus)

int straight(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns an int in (4000,5000] if there is a straight and 0 if not (score is base score + tiebreaker bonus)

int twoPair(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns an int in (2000,3000] if there is a two pair and 0 if not (score is base score + tiebreaker bonus)

int highCard(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns an int within (0,1000] if there is a high card and 0 if not (score is base score + tiebreaker bonus)

int kicker(PILE unusedHand, int whichKicker)

- Arguments: list of cards in the hole cards + community cards that were not used in a ranked hand; value (1, 2, or 3) indicating which kicker is being looked for (e.g. two of a

kind would call this function for all three possible values; two pair would only call it with whichKicker = 1)

- Result: returns the enum value of the highest unused card if whichKicker = 1, second highest for whichKicker = 2, and third highest for whichKicker = 3

3.3 Detailed Description of Communication Protocol

The poker game uses a client-server model in which all player actions/decisions are sent from the specific client to the server. All game-state updates and next steps stem from the server back to the client. The server will be in charge of all game flow, card dealing, pot updates, turn order, and round transitions, so that clients can easily display information and send necessary input. This helps to prevent clients from changing game state and keeping the game flow synchronized.

Function calls

client functions

- socket(), gethostbyname(), connect(), bzero(), read()

server functions

- socket(), bzero(), htons(), bind(), listen(), accept(), read()

Communication will be separated into login and game data. The login messages are primarily for the first player or host of the game, while in comparison game messages are for active play to transmit updates concerning player turns, bets, community cards, and round changes.

Protocol flags and settings

- packageType for both gameState and LoginInfo so the receiver identifies incoming buffers
- LoginInfo includes isHost, playerType, numPlayers, playerNum, password, & playersFound

4. Development Timeline

4.1 Partitioning of Tasks

Week 6	Week 7	Week 8	Week 9	Week 10
Create user manual	Create software specification Divide work	Create alpha version of code with text-based interface	Create beta version of code Fix any bugs	Create final version of the code

	amongst team members	Focus on relaying messages to clients via the server	with alpha version	
--	----------------------	--	--------------------	--

4.2 Team Member Responsibilities

Sanaa Bebal - clientNetwork.cpp / serverNetwork.cpp / clientGUI.cpp / serverGUI.cpp / clientMain.cpp

Ava Chinn - logic.cpp / cards.cpp / botDecisions.cpp / botPlayer.cpp

Alonso De La Cruz - logic.cpp / cards.cpp / botDecisions.cpp / botPlayer.cpp

Estrella Distancia - clientMain.cpp / serverMain.cpp / serverGUI.cpp

Lauren Howe - dealer.cpp / gamestate.cpp / cards.cpp / analysis.cpp

Samuel Kim - logic.cpp / gamestate.cpp / analysis.cpp

David Lee - gamestate.cpp / logic.cpp / analysis.cpp / dealer.cpp

5. References and External Libraries

<https://canvas.eee.uci.edu/courses/81045/pages/projects>

Used for specification of the rules and requirements of Anteater Poker

https://www.linuxhowtos.org/C_C++/socket.htm

Used as a reference for clientNetwork.cpp and serverNetwork.cpp

<https://github.com/jmqueen01/EECS22L-chess/blob/main/README>

Used as a reference for installation Linux code

Contact Information

If you have any questions, comments, or concerns about this software, please contact the creators using the following email addresses:

- sbebal@uci.edu
- amchinn@uci.edu
- alonsojd@uci.edu
- edistanc@uci.edu
- lehowe@uci.edu
- samuek16@uci.edu
- davijl18@uci.edu

Note: Any email pertaining to this software sent to one or more of the addresses above may be forwarded to any or all of the other emails above.

Terms and Conditions

Copyright

© 2026 All In. All rights reserved.

The unauthorized sale or distribution of this software is prohibited.

Disclaimer of Warranty

THIS SOFTWARE IS PROVIDED “AS IS” WITHOUT ANY EXPRESS OR IMPLIED WARRANTY OF ANY KIND.

Index

All in - 5	Flop - 4	severMain.cpp - 10, 19
Analysis.cpp - 12, 20	Flush - 4	serverNetwork.cpp - 10, 20
Anteater card - 3	Fold - 5	Showdown - 4
Anteater Pair - 4	Four of a kind - 4	Side pot - 3
Anteater Poker- 3	Full house - 4	Straight - 4
Bet - 5	Game end - 3	Straight flush - 4
Blinds - 3	Gamestate.cpp - 9, 17	struct gameState - 5
Board -3	Hand - 3	struct loginInfo
botDecisions.cpp - 11, 19	High card - 4	struct Player - 6
botPlayer.cpp - 9, 18	Hole cards - 3	Suit - 3
Call - 5	Kicker - 3	Three of a kind - 4
Cards.cpp- 9, 19-20	Logic.cpp - 9, 17	Turn - 4
Check - 5	Pair - 4	Two pair - 4
Classes - 6	Pot - 3	Value - 3
clientGUI.cpp - 9, 16	Preflop - 4	
clientMain.cpp - 9, 16	Raise - 5	
clientNetwork.cpp - 9, 17		
Community cards - 3	River - 4	
Dealer.cpp - 10, 19	Round - 3	
Dealer - 3	Royal flush - 4	
Five-card hand - 3	serverGUI.cpp - 10, 19	